Embedded Systems: 2-Month Intensive Plan

C, ARM Cortex-M, peripherals, RTOS, and CAN — built around real projects and a GitHub portfolio. 4–6 hours/day, ~240–350 total hours.

How to use this plan

Two months at 4–6 hours a day is enough for genuine depth on the listed topics, but only if you resist chasing tutorials. The plan is structured around one principle: **every week ends with a thing you built and committed to GitHub.** No exceptions. If a week ends without a working artifact, stay on it until there is one.

About 60% of your time should be spent on *your own code that doesn't work yet*, not consuming material. Buy the hardware in week 1 — software-only embedded is fake embedded.

Hardware Shopping List

Order everything in week 1 — don't wait until you 'need' it. Shipping time has killed more learning plans than anything else.

Core boards (~$30)

- **NUCLEO-F446RE** (~$15–20) — primary STM32 board. Cortex-M4F, 180 MHz. Mouser, DigiKey, or Amazon. NUCLEO-F411RE is a fine substitute.
- **Raspberry Pi Pico 2** (~$5) — secondary board for protocol experiments. Get the 'H' version with pre-soldered headers.

Debug and measurement tools (~$30–80)

- **8-channel USB logic analyzer** (~$10–15) — search 'USB Logic Analyzer 24MHz 8CH'. Works with free PulseView/Sigrok. Do not skip this.
- **Multimeter** (~$30–60) — Fluke 101, AstroAI WH5000A, or similar. Continuity, DC voltage, current. You do not need a high-end one.
- **Optional USB oscilloscope** (~$50–100) — Hantek 6022BE is the cheap option. Skippable for these two months.

Breadboarding supplies (~$25)

- Solderless breadboard, full-size (~$8)
- Jumper wire kit, male-male and male-female (~$8)
- LEDs, 220Ω and 10kΩ resistor packs, tactile push buttons (~$5)
- Decoupling capacitors: 100nF ceramic, 10µF electrolytic (~$4)

Sensors and peripherals (~$15–25)

- **MPU-6050 IMU breakout** (~$3) — I2C device for sensor projects
- **BME280 or BMP280 breakout** (~$5) — SPI sensor (BME280 supports both)
- **MCP2515 CAN modules with TJA1050 transceiver** (~$5 each, **buy two**) — CAN needs at least two nodes
- Header pins, if breakouts arrive un-soldered (~$3)

Cables (~$10)

- Two good USB cables — the ones included with boards are often flaky
- Optional ST-Link V2 clone (~$5) — Nucleo has one built in, so skip unless you have a reason

Total: ~$110–160 depending on scope and multimeter choice.

> **Ordering strategy:** Place two orders on day 1. **Order A (fast):** Nucleo, USB cable, breadboard kit, multimeter from Amazon/DigiKey/Mouser. **Order B (slow):** Logic analyzer, MPU-6050, BME280, 2× MCP2515, Pi Pico 2 from AliExpress. The slow order lines up with when you actually need those items.

When each item is first used

Week	New items in use
1	Laptop only — pure C on your computer
2	Nucleo, breadboard supplies, multimeter
3	Logic analyzer
4	MPU-6050, BME280
5	No new hardware — linker scripts and memory layout
6	No new hardware — RTOS port of existing project
7	2× MCP2515 CAN modules, 120Ω termination resistors, Pi Pico 2
8	No new hardware — capstone uses everything you already have

Month 1 — C, the Toolchain, ARM, and Core Peripherals

Week 1 — C, deeply

Most embedded resources assume you know C. You probably don't, not at the level you need. This week is uncomfortable but pays compound interest for years.

Spend serious time on: pointers (including pointer arithmetic and `*p++` vs `(*p)++`), arrays vs pointers (they are *not* the same thing), structs and padding/alignment, bitwise operators and bit fields, `volatile` (read 'Volatile — Multithreaded Programmer's Best Friend' by Andrei Alexandrescu), `const`, `static`, the preprocessor, and `extern`.

Resources:

- **Beej's Guide to C Programming** — free at beej.us/guide/bgc. Read cover to cover, do every exercise.
- **Modern C by Jens Gustedt** — free PDF for anything Beej skims.
- **'Memory Layout of C Programs'** on GeeksForGeeks — short and important.
- Daily output: write small C programs on your laptop, compile with `gcc -Wall -Wextra -Wpedantic`. Linked list, ring buffer, struct-based FSM. Push to a GitHub repo `c-foundations`.

End of week artifact: a ring buffer implementation in pure C with a test harness, on GitHub. Ring buffers come up in every embedded interview.

Week 2 — STM32 toolchain, ARM Cortex-M, your first bare-metal blink

Install **STM32CubeIDE** (free from st.com). Learn what it is: an Eclipse-based IDE wrapping the ARM GCC toolchain, OpenOCD, and ST-Link drivers. Understanding what CubeIDE is hiding makes the transition to VS Code + PlatformIO trivial later.

ARM Cortex-M topics: memory map (Flash, SRAM, peripheral regions), the vector table, the boot sequence (Reset_Handler), CMSIS, and the difference between the M0/M3/M4/M7 cores.

Resources:

- **Israel Gbati's 'Embedded Systems Bare-Metal Programming Ground Up (STM32)'** on Udemy. Wait for the $15 sale (weekly). Before he writes any peripheral code, *you* find the relevant RM section and try it yourself first.
- **STM32F446 Reference Manual (RM0390)** — your bible for 8 weeks. Learn to navigate it, don't read it linearly.
- **STM32F446 Datasheet** — separate document. Datasheet = electrical, RM = peripheral behavior.
- **Joseph Yiu's 'The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors'** — skim memory model, exception model, NVIC chapters.

Topics: GPIO (input, output, alternate function, pull-up/down), the RCC (clock tree — really understand this, 50% of beginner bugs live here), basic interrupts (NVIC, EXTI for button press).

End of week artifact: a bare-metal GPIO project (LED blink + button-triggered LED toggle via EXTI interrupt) using **direct register writes only** — no HAL, no CubeMX-generated init. README explains every register write with a reference to the RM0390 section number.

Week 3 — Timers, UART, interrupts deeply, and real debugging

Continue Gbati. Add **Phil's Lab on YouTube** (@PhilsLab) — his videos use VS Code + PlatformIO, closer to professional practice.

Topics: general-purpose timers (basic time base, PWM, input capture), USART (polling, then interrupt-driven), and **the debugger** — breakpoints, stepping, watch expressions, SFR view, memory view. Spend at least one full day on the debugger. Most beginners use printf debugging forever because they never learned the tool.

Logic analyzer first use: capture UART output, decode it in PulseView, verify bit timing matches baud rate (one bit period at 115200 baud ≈ 8.7 µs — measure it). This is the moment things click.

End of week artifact: a UART command-line interface — STM32 receives commands like `led on`, `pwm 50` over serial and responds. Interrupt-driven UART (not polling). README includes PulseView decode screenshot.

Week 4 — ADC, I2C, SPI, and a real sensor

Topics: ADC (single, continuous, then with DMA), I2C (read from MPU-6050), SPI (read from BME280 in SPI mode), and DMA broadly (everywhere, undertaught).

The I2C work is most important — use the logic analyzer to capture every transaction. You should be able to identify start condition, 7-bit slave address, R/W bit, ACK bits, and data bytes by eye. PulseView will decode; first try to read raw, then verify with the decoder.

End-of-month-1 artifact (big one): a **USB-serial IMU data logger**. Reads accelerometer and gyro from MPU-6050 over I2C at 1 kHz, timestamps each sample using a hardware timer, streams data out UART at 921600 baud, with a small Python script that captures and plots it. Direct register access (HAL acceptable for I2C/SPI if it's killing you, but I2C is doable bare-metal and worth doing once). Include: code, wiring photo, logic analyzer captures of I2C, plots showing tap/rotation, README explaining architecture. This project alone, done well, makes you more capable than 70% of new graduates.

> **Background reading throughout month 1:** *Making Embedded Systems* by Elecia White, 2nd edition (2024). One chapter every couple of days. The closest thing the field has to a canonical text.

Month 2 — Linker Scripts, RTOS, CAN, and the Portfolio Project

Week 5 — Linker scripts, memory layout, undefined behavior

The 'looks small, is huge' week. You go from 'the code just runs' to 'I know exactly where every byte lives.'

Topics:

- Linker script (`.ld`) — sections (`.text`, `.rodata`, `.data`, `.bss`), vector table placement, MEMORY and SECTIONS blocks, why `.data` is copied from Flash to SRAM at startup.
- Startup file (`startup_stm32f446xx.s`) — what Reset_Handler does, how main() gets called.
- Stack and heap placement, stack overflow detection.
- Map files — generate one (`-Wl,-Map=output.map`) and *read it*. Find any symbol's address.
- Undefined behavior in C: read John Regehr's 'A Guide to Undefined Behavior in C and C++' (3 parts), skim CERT C high-severity rules, read 'What Every C Programmer Should Know About Undefined Behavior' on the LLVM blog.

Exercise: modify the linker script to put a specific variable at a specific Flash address, verify with the map file, verify again by reading memory with the debugger. Then reduce stack size and intentionally cause a stack overflow — see what happens (usually a HardFault, learn to debug it using CFSR/HFSR registers).

End of week artifact: a 'system info' firmware that reports over UART at boot: total Flash used, total RAM used, stack high-water mark, vector table location, and a deliberately placed string at a fixed Flash address. README explains the linker script changes.

Week 6 — FreeRTOS fundamentals

Pick one, not both. Recommendation: **FreeRTOS first**, Zephyr later. FreeRTOS is smaller, easier, and concepts transfer directly. Zephyr's build system (West, Kconfig, devicetree) is a real curve on top of the RTOS itself.

Resources:

- **'Mastering the FreeRTOS Real Time Kernel'** by Richard Barry — free PDF at freertos.org. Read chapters 1–6 carefully.
- **DigiKey 'Introduction to RTOS' YouTube series** by Shawn Hymel — 10 episodes, ~15 min each, exceptionally clear. Watch all.
- **Memfault Interrupt blog** (interrupt.memfault.com) — search FreeRTOS posts, especially debugging.

Topics: tasks, scheduler, priorities, queues, semaphores (binary and counting), mutexes, `vTaskDelay` vs `vTaskDelayUntil`, stack sizing, inspecting RTOS state in the debugger (task list, stack high-water marks).

End of week artifact: port the IMU logger from week 4 to FreeRTOS. One task samples the IMU at 1 kHz. A second computes a complementary filter for roll and pitch. A third formats and sends data over UART. Communication via FreeRTOS queues. README discusses stack sizing decisions and shows debugger screenshots of the task list.

Week 7 — CAN bus, deeply

High-leverage week. Almost no hobbyist learns CAN properly, and it's the protocol of every car, every industrial robot, most professional motor controllers.

Topics: CAN physical layer (differential signaling, termination resistors), frame format (standard vs extended IDs, DLC, CRC), arbitration, bit timing (sync, prop, phase1, phase2 — where everyone gets stuck), acknowledgment.

Resources:

- **CSS Electronics 'CAN Bus Explained — A Simple Intro'** — best beginner CAN resource on the internet.
- **Vector CAN webinars on YouTube** — search 'Vector CAN basics'.
- **STM32F4 reference manual chapter on bxCAN** — your reference.

Hardware setup: wire two MCP2515 modules to two MCUs (STM32 + Pico, or two STM32s). CAN_H to CAN_H, CAN_L to CAN_L. Add 120Ω termination at each end of the bus — non-negotiable, CAN won't work without it.

End of week artifact: a **CAN bus sniffer + transmitter**. STM32 sends CAN frames with sensor data at 100 Hz. Pico receives and forwards over UART to laptop; Python script logs them. Add a filter: only forward frames with IDs in a certain range. Capture CAN traffic with the logic analyzer (500 kbps is within range) and include decoded captures in the README. Portfolio-grade project.

Week 8 — Portfolio polish and the capstone

Days 1–3 — the capstone: build a **multi-protocol sensor node**. STM32 reads IMU over I2C, BME280 over SPI, runs FreeRTOS with separate tasks, publishes combined data over **both** UART and CAN simultaneously, with configurable rate via UART commands. Pulls together every skill from the two months. Use the logic analyzer to verify both buses are clean and not interfering. Capture LA traces of all three protocols running at once.

Days 4–5 — GitHub cleanup. For each repo:

- Real README: what the project does, hardware required, how to build, how to run.
- Wiring photo or schematic (KiCad's free schematic editor; hand-drawn is fine).
- Logic analyzer screenshots showing the protocol actually working.
- Short 'what I learned' or 'bugs I fixed' section — what hiring managers actually read.
- Pin your three best repos on your GitHub profile.

Days 6–7 — write one technical blog post. 1500–2500 words on the most interesting bug you fixed. Medium, Hashnode, or GitHub Pages. Model tone on Memfault or Phil's Lab. Title it something specific like 'Why my I2C reads were returning 0xFF until I fixed the pull-ups' rather than something generic. This post will outperform a resume.

Habits That Determine Whether This Works

Keep an engineering log

Plain text file, one entry per day: what you tried, what worked, what didn't, what the scope or logic analyzer showed. This is what senior engineers do — and 80% of how they became senior. When you debug a tricky problem in week 7, your week 3 notes will save you an hour.

Use git like a professional from day 1

Real commit messages. If your history is a row of 'wip' you haven't actually learned the tool.

Resist switching chips

You'll be tempted to try ESP32, then nRF52, then RISC-V. Don't. Two months on STM32 with the same reference manual is how you build the navigation skill that transfers to any chip later. Depth, not breadth.

On AI use

Use AI as a *tutor* for understanding, not a code generator. 'Explain why this RCC config is wrong' is great. 'Write me a UART driver' is poison at this stage — the typing of the driver *is* the learning. If you let AI write your peripheral code, you'll pass the project but fail the interview.

The wall is coming

Somewhere around week 3 or week 6, something will break in a way that makes you feel like you've made no real progress and aren't cut out for this. That feeling is the work. When it happens, take a single day off, then return to the exact same problem rather than switching to feel productive. The people who push through that week are the ones who become embedded engineers.

> Two months of this, done seriously, puts you in a different category than 95% of mechatronics students. The goal isn't to 'finish the syllabus' — it's that at the end, you can hold a 30-minute conversation with a senior firmware engineer about your projects and have them lean in. Build for that conversation.